

GPU Kernel Programming: Performance Guidelines and Patterns

Advanced GPU Short Course

Piotr Jarosik, Marcin Lewandowski, Piotr Karwat

us4us Ltd.

Agenda

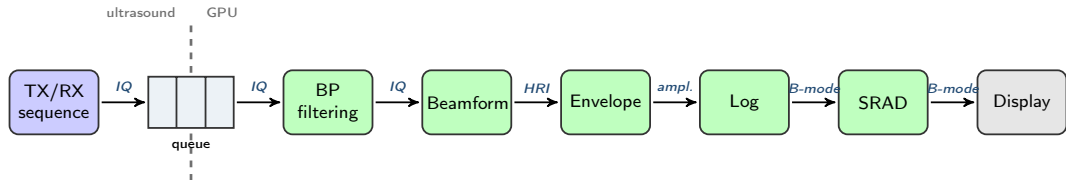
Performance Guidelines

- Memory access patterns
- GPU libraries
- Control flow
- Multiprocessor occupancy

HPC Patterns

- Stencil
- Iterative stencil

Example: Ultrasound B-mode image reconstruction



Performance Guidelines

Performance guidelines

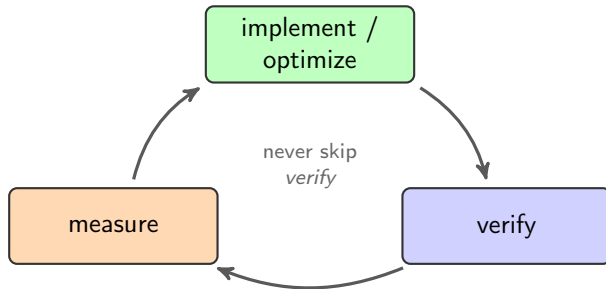
In this section we will consider the following aspects of optimizing CUDA kernels:

- **memory access patterns:** *memory coalescing* for the best throughput,
- **control flow:** how code branching can affect the performance,
- **multiprocessor occupancy:** experimenting with different block sizes,
- **GPU libraries:** the fastest kernel is typically the one you do not write – what libraries are available and how to use them.

You will also learn how profiling metrics can tell us where the performance issues are.

This section is mostly based on the content available in the *CUDA C++ Best Practices Guide*,
<https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/>

Iterative improvements



The kernel's performance can be evaluated using:

- **NVIDIA:** Nsight Systems (timeline, whole application), Nsight Compute (one kernel, metrics)
- **AMD:** rocprof (counters, traces), rocprof-sys (formerly Omnitrace, timeline), rocprof-compute (formerly Omnipperf, kernel analysis)

Performance metrics

what you suspect	NVIDIA (ncu)	AMD (rocprof)
memory access pattern	gld_efficiency, gst_efficiency, sectors per request	FETCH_SIZE, WRITE_SIZE, MemUnitStalled
cache behaviour	l1tex__t_sector_hit_rate, lts__t_sector_hit_rate	TCP_CACHE_HIT, TCC_HIT_sum
divergence	branch_efficiency, warp_execution_efficiency	SQ_WAVE_CYCLES, SQ_INSTS_VALU
occupancy	achieved_occupancy, registers/thread	SQ_WAVES, GRBM_GUI_ACTIVE
arithmetic	sm__throughput, FLOP counters	SQ_INSTS_VALU, VALUUtilization

Pick the metric *after* forming a hypothesis (the list is quite long).

Profiler performance metrics: example output

Which kernel should we optimize?

First, measure the performance (Nsight Systems)

```
$ nsys profile -o fir ./fir_filter -n=4194304 -taps=64 -width=1024
$ nsys stats -report cuda_gpu_kern_sum fir.nsys-rep
```

Time (%)	Instances	Med (ns)	Name
65.0	56	69 567 598	convolve1d_axis0
15.0	120	4 159 965	convolve1d_global
8.0	56	4 201 808	convolve1d_shared
7.0	56	4 162 974	convolve1d_axis1
4.0	56	2 559 760	convolve1d_shared_const

Then ask *why*, for that one kernel (Nsight Compute):

```
$ ncu -kernel-name regex:convolve1d_axis -metrics \
    smssp__sass_average_data_bytes_per_sector_mem_global_op_ld.pct,\
    smssp__sass_average_data_bytes_per_sector_mem_global_op_st.pct ./fir_filter ...

convolve1d_axis0    ..._global_op_ld.pct 12.52 %    ..._global_op_st.pct 12.50 %
convolve1d_axis1    ..._global_op_ld.pct 70.70 %    ..._global_op_st.pct 100.00 %
```

CUDA Best Practices Guide: memory coalescing

The concurrent accesses of the threads of a warp will coalesce into a number of transactions equal to the number of **32-byte transactions** necessary to service all of the threads of the warp.

Implement a GPU kernel that only loads *useful* data from global memory. We can achieve this with **coalesced memory accesses**:

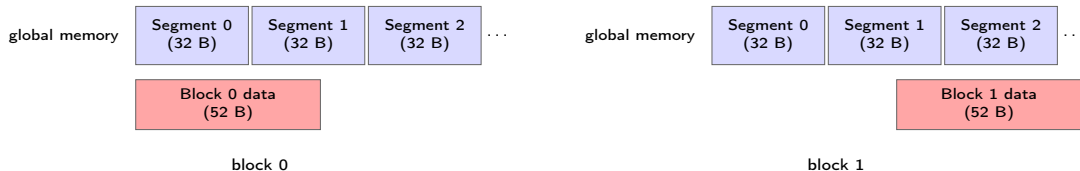
- the number of threads per block should be a **multiple of 32 threads**,
- sequential threads in a warp access memory that is sequential, with no offset.

Impact of misaligned accesses

CUDA Best Practices Guide

The number of threads per block should be a **multiple of 32 threads**, because this provides optimal computing efficiency and facilitates coalescing.

Typically, we assign one thread to read/write one output voxel/texel/sample/etc.



Impact of misaligned accesses: experiments

Is there any observable performance difference between `log_compression` launched with **256** and **261** threads in a block?

block size	kernel time	sector efficiency	L2 read sectors
256 (aligned)	9.4 ms	100.0 %	2.10 M
261 (misaligned)	9.6 ms	80.6 %	2.15 M (+2.3 %)

The difference is rather minor – each warp moves 20 % more sectors, but adjacent warps **reuse the cached data** their neighbors fetched:

CUDA Best Practices Guide

So while the impact is still evident it is not as large as we might have expected. It would have been more so if adjacent warps had not exhibited such a high degree of reuse of the over-fetched cache lines.

Still, **setting a block size to a multiple of warp size** is a good rule of thumb.

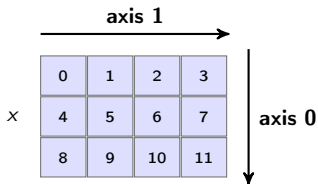
Impact of strided accesses

Guideline

Threads with successive identifiers read should the data from successive memory areas:

$$x = \text{data}[(\text{some custom offset}) + \text{threadIdx.x}]$$

Important when working with n-d array – the choice of the axis, along which a specific operation is performed, can affect effective bandwidth.



NumPy / C store arrays in **row-major** order:

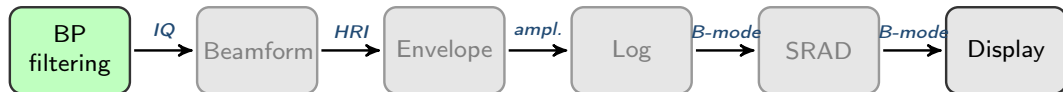


MATLAB / Fortran store them **column-major**:



The same code walking “the fast axis” therefore means a *different* axis in the two conventions.

Example: Bandpass filter (convolution)



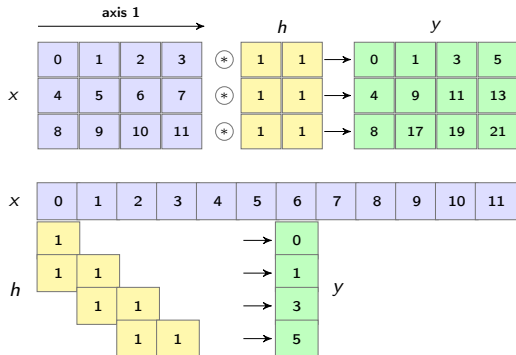
$$y[i] = \sum_{j=0}^{N-1} x[i + o - j] h[j]$$

Example: Bandpass filter (convolution)

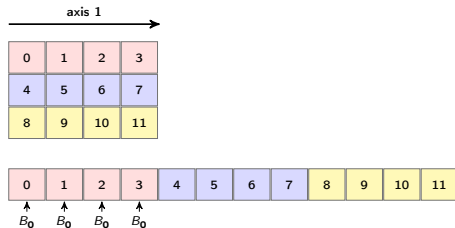
```
// along axis 0
int sample = blockIdx.x*blockDim.x + threadIdx.x; // row
int scanline = blockIdx.y*blockDim.y + threadIdx.y; // column
for(int j = 0; j < n; ++j) {
    int k = sample + o - j;
    // TODO Simplified for clarity only. Remember to add guard here!
    value += x[k*width + scanline]*h[j];
}
y[sample*width + scanline] = value;
```

```
// along axis 1
int sample = blockIdx.x*blockDim.x + threadIdx.x; // column
int scanline = blockIdx.y*blockDim.y + threadIdx.y; // row
for(int j = 0; j < n; ++j) {
    int k = sample + o - j;
    // TODO simplified for clarity only. Remember to add guard here@!
    value += x[scanline*width + k]*h[j]
}
y[scanline*width + sample] = value;
```

Example: Bandpass filter – Convolution along axis 1

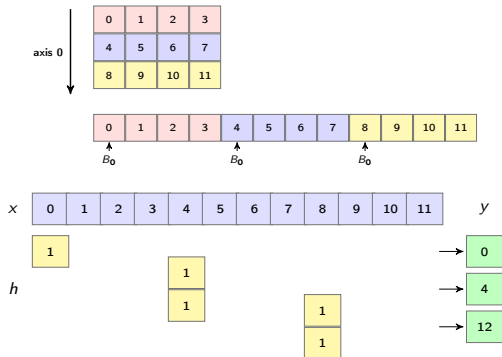
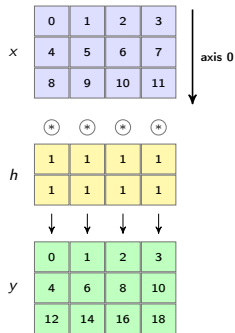


y[0], y[1], y[2] and y[3] are computed by threads with
threadIdx.x equal 0, 1, 2 and 3.



If the colors red, blue and yellow represent successive segments in the GPU memory, all readings for the computation of the first row fit in one block.

Example: Bandpass filter – Convolution along axis 0



$y[0]$, $y[1]$ and $y[2]$ are computed by threads with `threadIdx.x` equal 0, 1 and 2. The stride is expected to be much larger for the convolution along axis 0. Will it impact the bandwidth?

GPU libraries: do not reinvent wheel

Guideline

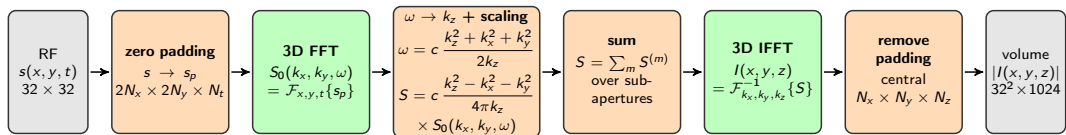
Before implementing a custom kernel, check whether there is a functionality ready to use in the vendor library.

A GPU library is usually optimized per architecture, and handles the edge cases you might have not thought of yet.

what you need	CUDA	HIP (AMD)
FFT	cuFFT	hipFFT / rocFFT
dense linear algebra	cuBLAS	hipBLAS / rocBLAS
sparse linear algebra	cuSPARSE	hipSPARSE
scan, reduce, sort, etc	CUB	rocPRIM
linear algebra solvers	cuSOLVER	hipSOLVER
image / signal primitives	NPP	rocAL, MIVisionX

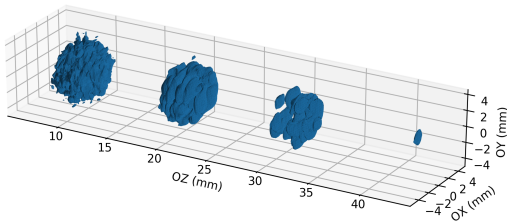
Example: 3D volume reconstruction (HSDI)

Hybrid Spectral-Domain Imaging (HSDI)



cuFFT / hipFFT – a library call

custom CUDA kernels



Why the frequency domain, for 3D

Time domain (delay-and-sum)

For every voxel, sum over every receiving element:

$$O\left(\underbrace{N_x N_y N_z}_{\text{voxels}} \times \underbrace{M^2}_{\text{channels}}\right)$$

A 32×32 aperture into a $32 \times 32 \times 1024$ volume is $1.0 \cdot 10^6$ voxels $\times$ 1024 channels $\approx 10^9$ accumulations – *per transmit*.

- The advantage grows with the aperture: doubling M costs the time domain $4\times$, the frequency domain almost nothing.
- This is what makes real-time 3D plausible on a single card.

Frequency domain (HSDI)

A few 3D transforms plus one pass over the spectrum:

$$O(N \log N), \quad N = 64 \cdot 64 \cdot 4096$$

The channel sum becomes a **superposition of spectra** – addition, not a loop. And the transforms are a library call.

How to use cuFFT: setup

```
#include <cuFFT.h>                                // link with -lcufft

cufftComplex *d_cube, *d_spec, *d_padded;          // interleaved complex (float2)
cudaMalloc(&d_cube, nxz*nyz*nt*sizeof(cufftComplex)); // (2Nx, 2Ny, Nt)
cudaMalloc(&d_spec, nxz*nyz*nz*sizeof(cufftComplex)); // (2Nx, 2Ny, Nz)
cudaMalloc(&d_padded, nxz*nyz*nz*sizeof(cufftComplex));

// one plan per transform size and type, created ONCE, reused every volume
cufftHandle plan_fwd, plan_inv;
cufftPlan3d(&plan_fwd, nxz, nyz, nt, CUFFT_C2C);    // 64 x 64 x 4096
cufftPlan3d(&plan_inv, nxz, nyz, nz, CUFFT_C2C);    // 64 x 64 x 1024
// ... reconstruct volumes ...
cufftDestroy(plan_fwd); cufftDestroy(plan_inv);
```

- **plan**: size, type (C2C, R2C, Z2Z for double), batch – the algorithm is chosen *here*, plus a work area on the device
- layout: row-major, the **last** dimension is contiguous

How to use cuFFT: one volume

```
for (int em = 0; em < nem; ++em) {                                // RX sub-apertures
    zero_pad<<<grid_pad, block>>>(d_cube, d_rf + em*rf_size, ...);
    cufftExecC2C(plan_fwd, d_cube, d_cube, CUFFT_FORWARD); // in place
    interp_scale<<<grid_int, block>>>(d_spec, d_cube, ...,
                                      /*accumulate=*/em > 0); // + sum
}
cufftExecC2C(plan_inv, d_spec, d_padded, CUFFT_INVERSE); // out of place
// cuFFT does not normalize: scale by 1/(nxz*nyz*nz) while cropping
remove_padding<<<grid_crop, block>>>(d_volume, d_padded, ...,
                                     1.0f/(nxz*nyz*nz));
```

- **two library calls**, everything in between is ours: padding, $\omega \rightarrow k_z$ migration + scaling (+ the sub-aperture sum), cropping
- `cufftExec*` is asynchronous, like a kernel launch
- no `fftshift` in cuFFT: `interp_scale` reads the spectrum with shifted indices instead

Control flow: how code branching affects the performance

Due to SIMT architecture of the CUDA multiprocessors, it is recommended to avoid different paths within the same warp.

CUDA Best Practices Guide

Flow control instructions (`if`, `switch`, `do`, `for`, `while`) can significantly affect the instruction throughput by causing threads of the same warp to diverge; that is, to follow different execution paths. If this happens, the different execution paths must be executed separately; this increases the total number of instructions executed for this warp.

Control flow: over-complicated example

```
// the easy way  
int r = i % 8;  
c[i] = r*a[i] + b[i];
```

```
// the overcomplicated way  
if(i % 8 == 0) {  
    c[i] = b[i];  
}  
else if(i % 8 == 1) {  
    c[i] = 1*a[i] + b[i];  
}  
// ...  
else if(i % 8 == 7) {  
    c[i] = 7*a[i] + b[i];  
}
```

kernel	time	branch_efficiency
simple	50.0 μ s	100.00%
overcomplicated	112.1 μ s	58.82%

branch_efficiency: ratio of non-divergent branches to total branches expressed as percentage.

Of course, the above example has been artificially complicated just to show the effect of complex kernel logic on the kernel's performance.

Control flow: in our example

threadIdx.x runs along z: a warp is 32 neighbouring depths of one line, and the only branch in the inner loop is a depth test.

```
for(int iRx = 0; iRx < nRx; ++iRx) {  
    // time of flight: the same code for every lane  
    if(iSamp < 0.0f || iSamp >= nSamp-1) { continue; } // outside the recording  
    // accumulate  
}
```

	branch_eff.	warp_exec_eff.
recording covers the image (--nsamp=1280)	100.00%	100.00%
image deeper than it (--nsamp=500)	100.00%	94.29%

τ grows monotonically with z , so the guard splits a warp at **one** boundary – and only where the image outruns the recording.

Multiprocessor occupancy: thread block size

Recall that: *the number of threads per block should be a multiple of 32 threads*, because this provides optimal computing efficiency and facilitates coalescing.

CUDA Best Practices Guide: rules of thumb

1. Threads per block should be a **multiple of warp size** to avoid wasting computation on under-populated warps and to facilitate coalescing.
2. A **minimum of 64 threads** per block should be used, and only if there are multiple concurrent blocks per multiprocessor.
3. Between **128 and 256 threads** per block is a good initial range for experimentation with different block sizes.
4. Use several smaller thread blocks rather than one large thread block per multiprocessor if latency affects performance. This is particularly beneficial to kernels that frequently call `__syncthreads()`.

Multiprocessor occupancy: experiments

achieved_occupancy: ratio of the average active warps per active cycle to the maximum number of warps supported on a multiprocessor.

block size	add_vectors time	achieved_occupancy
16	126.4 μ s	0.182
256	50.0 μ s	0.874
1024	49.3 μ s	0.813

Finding the right number of threads per block requires some experimentation, but generally **128 or 256 threads is a good starting point.**

HPC Patterns

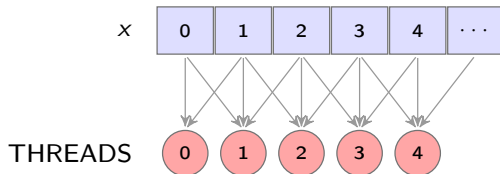
Task

$$\text{out}[i] = f(\text{neighbourhood of in}[i])$$

- Examples: 1D/2D convolution: FIR filter (RF/IQ), smoothing, etc.
- Neighbouring threads read **overlapping** input windows
- Technique: **memory tiling**

Stencil: 1D convolution

Let's layout calculations for the 1D conv.

$$\begin{aligned}y[0] &= h[0]*x[1]+h[1]*x[0] \\y[1] &= h[0]*x[2]+h[1]*x[1]+h[2]*x[0] \\y[2] &= h[0]*x[3]+h[1]*x[2]+h[2]*x[1] \\y[3] &= h[0]*x[4]+h[1]*x[3]+h[2]*x[2] \\y[4] &= h[1]*x[4]+h[2]*x[3]\end{aligned}$$


Multiple threads read the same data from the global memory many times:

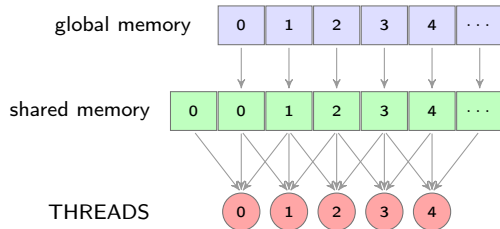
- reads $x[0]$ and $x[4]$ from the global memory twice,
- reads $x[1]$, $x[2]$ and $x[3]$ three times.

Stencil: the shared memory pattern

A typical CUDA shared memory usage pattern is as follows.

Each thread i in a block:

1. reads data from the global to shared memory,
2. waits until all other threads in the block finish 1.,
3. perform calculations using data available in the shared memory.



Stencil: 1D convolution

We can apply the above pattern to the `convolve_gpu_kernel` implementation:

- let's declare that each block has shared memory for $B + N - 1$ samples, where B is the size of the threads block, N is the length of the filter,
- for each thread i in a block:
 1. read a portion of data from global to shared memory,
 - start with $k = i + o - (N - 1)$,
 - read from global to shared memory: $x[k]$, $x[k + B]$, $\dots$, and so on, until you hit the end of the shared memory area,
 - if $k + jB < 0$ or $k + jB \geq m$ for any j , write 0 to shared memory at position $i + jB$, otherwise write $x[k + jB]$,
 2. wait until all threads in the block finish 1.,
 3. compute $y[i]$.

Stencil: step 1, read from global to shared memory

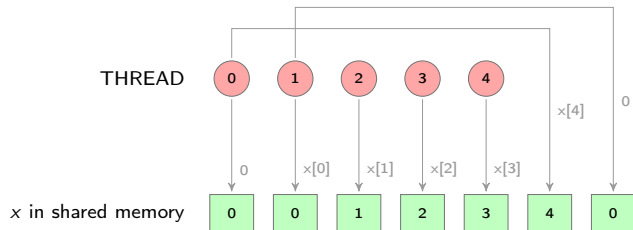
Our toy example, assume $B = 5$ (one block computes all the output samples), $N = 3$, $o = 1$:

$x = [0, 1, 2, 3, 4]$

$h = [0, 1, 2]$

$\text{shared_memory} = [0, 0, 0, 0, 0, 0, 0]$

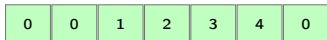
(note: an uninitialized shared memory may contain any arbitrary values, we used zeros just for clarity).



Stencil: steps 2 and 3

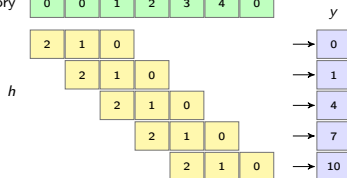
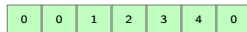
Step 2. Wait until threads finish the above procedure. At this point:

x in shared memory



Step 3. Compute y:

x in shared memory



Each $x[i]$ was read from the global memory exactly once.

Stencil: tile + halo

```
#define R 8
#define BLOCK 256
__constant__ float h[2*R + 1];

__global__ void fir_tiled(float *y, const float *x, int n) {
    __shared__ float tile[BLOCK + 2*R];
    int i = blockIdx.x*BLOCK + threadIdx.x;
    int t = threadIdx.x + R;
    tile[t] = (i < n) ? x[i] : 0.0f;           // body
    if (threadIdx.x < R) {                    // halos
        int l = i - R, r = i + BLOCK;
        tile[t - R] = (l >= 0) ? x[l] : 0.0f;
        tile[t + BLOCK] = (r < n) ? x[r] : 0.0f;
    }
    __syncthreads();                         // no early return before this line
    if (i >= n) return;
    float acc = 0.0f;
    #pragma unroll
    for (int k = -R; k <= R; ++k) acc += h[k + R]*tile[t + k];
    y[i] = acc;
}
```

- global reads per block: $B(2R + 1) \rightarrow B + 2R$

Stencil: when does it pay off?

- reuse factor:
 - ▶ 1D filter: $2R + 1$ – L1 often captures it already → **measure**
 - ▶ $K \times K$ 2D stencil: K^2
 - ▶ GEMM: tile size
- cost:
 - ▶ shared memory per block → occupancy
 - ▶ halo overhead: $2R/B$

Iterative stencil

$$u^{n+1}[y, x] = f(u^n[y, x], u^n[y \pm 1, x], u^n[y, x \pm 1]), \quad n = 0, 1, \dots, N_{it} - 1$$

- stencil, repeated N_{it} times
- iteration $n + 1$ needs **all** of iteration $n \rightarrow$ **grid-wide barrier** = kernel boundary
- ultrasound: anisotropic diffusion (Perona–Malik, **SRAD**), finite-difference wave simulation, Jacobi solvers

Iterative stencil: the host loop

```
for (int it = 0; it < n_iter; ++it) {  
    diffusion_step<<<grid, block, 0, s>>>(v, u, w, h, lambda);  
    std::swap(u, v);           // ping-pong: never update in place  
}
```

- kernels in one stream run in order: end of kernel = barrier
- **double buffering** (ping-pong): in place → neighbours see old + new values
- `std::swap`: free, no copies

Iterative stencil: performance

- **memory-bound**: a few flops per 5 loads + 1 store
- **2D tiling** + 1-pixel halo
- **kernel fusion**: the steps of one iteration in one kernel
- **temporal blocking**: k iterations per launch, k -pixel halo
 - ▶ $k\times$ less DRAM traffic, redundant work at tile borders
- **launch overhead**: 1000 iterations on $256 \times 256 \rightarrow$ 1000 kernels of a few μs
 - ▶ CUDA / HIP **graph**, or cooperative kernel + `grid.sync()`

Example: SRAD

Speckle reducing anisotropic diffusion: smooth speckle, keep edges.

$$q_0^2 = \text{var}(J_{ROI}) / \text{mean}(J_{ROI})^2$$

$$d_k = J_k - J, \quad k \in \{N, S, W, E\}$$

$$q^2 = \frac{\frac{1}{2} \sum_k d_k^2 / J^2 - \frac{1}{16} (\sum_k d_k / J)^2}{(1 + \frac{1}{4} \sum_k d_k / J)^2}$$

$$c = \left[1 + \frac{q^2 - q_0^2}{q_0^2 (1 + q_0^2)} \right]^{-1}$$

$$J' = J + \frac{\lambda}{4} (c d_N + c_S d_S + c d_W + c_E d_E)$$

one iteration = 3 kernels:

1. ROI statistics – **reduction**
2. coefficient c – **stencil**
3. update J' – **stencil**, ping-pong

References

- *CUDA Best Practices Guide*, <https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/index.html>
- *Efficient Matrix Transpose in CUDA C/C++* (NVIDIA Devblog Post)
- *How to Access Global Memory Efficiently in CUDA C/C++ Kernels* (NVIDIA Devblog Post)
- M. Lewandowski, P. Jarosik, Y. Tasinkevych, M. Walczak, *Efficient GPU Implementation of 3D Spectral Domain Synthetic Aperture Imaging* – the HSDI pipeline of the last example (examples/04_hsdi_volume)
- *cuFFT / hipFFT* documentation, <https://docs.nvidia.com/cuda/cufft/>